Universidad Centroamericana José Simeón Cañas
Departamento de Electrónica e Informática.

Administración de bases de datos
Laboratorio 4 Ejercicios G2

Grupo 2 - AdventureWorks - Recursos Humanos

Base de datos a utilizar: AdventureWorks2022
Entorno: contenedor Docker `sqlserver-2025` de la clase (`localhost,1434`, login `sa`)

Indicaciones

Restaurar la base de datos **AdventureWorks** desde su archivo `AdventureWorks2022.bak` (el instructor indicará la ruta dentro del contenedor). Trabaja con el nombre `AdventureWorks2022`.

El laboratorio consta de 4 ejercicios prácticos. Debes elegir y resolver **3**. Los ejercicios son independientes entre sí.

Entrega un archivo `.sql` y el PDF `tu-carnet_Laboratorio4.pdf` con capturas. Activa siempre el plan de ejecución real (`Ctrl + M`) y `SET STATISTICS IO ON`.

En las tablas pequeñas el optimizador puede mantener un Scan aunque exista un índice. Si te ocurre, documenta las lecturas lógicas y explica la decisión del motor.

Trabaja siempre sobre una base recién restaurada y no modifiques los datos existentes, salvo donde el ejercicio lo pida.

Ejercicio 1 · Esquemas y secuencias

Problema. Recursos Humanos quiere mover la información de turnos a un esquema propio y llevar en él un historial de auditoría, cuyos identificadores los genere automáticamente una secuencia en lugar de escribirse a mano.

Requerimiento

1. Crear el esquema `Reportes` con propietario `dbo`.
2. Transferir la tabla existente `HumanResources.Shift` al nuevo esquema con `ALTER SCHEMA ... TRANSFER` y comprobar su nueva ubicación en el catálogo del sistema.
3. Crear la secuencia `Reportes.seq_auditoria_horarios` de tipo entero, que inicie en 100, avance de 1 en 1, no haga ciclo y reserve 20 valores en memoria (CACHE).
4. Crear una **nueva tabla** `Reportes.Historial` con las columnas `id_registro` (INT), `usuario` (VARCHAR(50)) y `fecha` (DATE). Configura `id_registro` para que, por defecto (DEFAULT), tome el siguiente valor de `Reportes.seq_auditoria_horarios`.
5. Insertar dos filas en `Reportes.Historial` indicando únicamente el usuario y la fecha, y consultar la tabla para verificar que la secuencia asignó los identificadores correctamente.

Salida esperada. La tabla `Shift` aparece en el esquema `Reportes` y ya no en el original, y la nueva tabla `Historial` queda creada en ese mismo esquema. La consulta al catálogo de secuencias muestra `seq_auditoria_horarios` con valor inicial 100 e incremento 1. Las dos filas insertadas reciben automáticamente los identificadores 100 y 101 (sin que los escribas en el `INSERT`), junto con el usuario y la fecha indicados.

Ejercicio 2 · Índices y planes de ejecución

Problema. Nómina necesita el directorio de empleados (PersonType igual a «EM») con su preferencia de correo y fecha de modificación, y la consulta recorre toda la tabla de personas.

```
SELECT BusinessEntityID, Title, EmailPromotion, ModifiedDate
FROM Person.Person
WHERE PersonType = N'EM';
```

Requerimiento

1. Ejecutar la consulta con el plan real y `SET STATISTICS IO ON`. Identificar el operador de acceso a la tabla y anotar las lecturas lógicas.
2. Crear el índice (agrupado o no agrupado) que corresponda a la columna de búsqueda `PersonType`. Justifica por escrito tu elección.
3. Repetir la consulta y comparar operador y lecturas lógicas.

Salida esperada. Antes del índice, el plan muestra un Clustered Index Scan (o Table Scan) con lecturas iguales al tamaño de la tabla. Después, el plan debería pasar a un Index Seek sobre el índice nuevo (con Key Lookup si pides columnas que el índice no contiene) y las lecturas lógicas deberían bajar de forma notable. Si el optimizador conserva el Scan, explica por qué con las filas estimadas y las lecturas.

Ejercicio 3 · Optimizando el Key Lookup con INCLUDE

Problema. Bienestar laboral busca a las personas por apellido y necesita además su preferencia de correo y la fecha de la última modificación.

```
SELECT LastName, FirstName, EmailPromotion, ModifiedDate
FROM Person.Person
WHERE LastName = N'Smith';
```

Requerimiento

1. Ejecutar la consulta con el plan real y `SET STATISTICS IO ON`. Localizar el Key Lookup y anotar las lecturas lógicas.
2. Crear (o reemplazar el índice existente `IX_Person_LastName_FirstName_MiddleName`, documentando tu decisión) un índice no agrupado que cubra la consulta: decide qué columna va como clave y cuáles como columnas incluidas (EmailPromotion y ModifiedDate deben quedar disponibles en el índice).
3. Repetir la consulta y comparar el plan y las lecturas lógicas con las del paso 1. Si conservas ambos índices, verifica en el plan cuál usa el optimizador.

Salida esperada. Al inicio, el plan muestra un Index Seek seguido de un Key Lookup unidos por Nested Loops. Con el índice cubierto desaparece el Key Lookup, queda un único Index Seek y las lecturas lógicas disminuyen. Explica en dos líneas por qué las columnas incluidas no forman parte de la clave de búsqueda.

Ejercicio 4 · Análisis de JOINs y rendimiento (avanzado)

Problema. La dirección quiere conocer, por departamento, cuántos empleados activos hay y su tarifa promedio, solo para quienes no reciben promociones por correo.

```
SELECT d.Name AS Departamento,
       COUNT(DISTINCT e.BusinessEntityID) AS Empleados,
       AVG(ph.Rate) AS TarifaPromedio
FROM HumanResources.Employee AS e
JOIN Person.Person AS p ON p.BusinessEntityID = e.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory AS edh ON edh.BusinessEntityID =
e.BusinessEntityID
JOIN HumanResources.Department AS d ON d.DepartmentID = edh.DepartmentID
JOIN HumanResources.EmployeePayHistory AS ph ON ph.BusinessEntityID = e.BusinessEntityID
WHERE edh.EndDate IS NULL AND p.EmailPromotion = 0
GROUP BY d.Name
ORDER BY TarifaPromedio DESC;
```

Requerimiento

1. Ejecutar la consulta con el plan real, `SET STATISTICS IO ON` y `SET STATISTICS TIME ON`.
2. Documentar las lecturas lógicas de cada tabla (Employee, Person, EmployeeDepartmentHistory, Department y EmployeePayHistory), el tiempo de CPU y el tiempo transcurrido.
3. Identificar el algoritmo de unión que usó el motor entre cada par de tablas (Nested Loops, Hash Match o Merge Join) y el operador de mayor costo porcentual en el plan.
4. Proponer y crear **un** índice que reduzca el costo de la operación más pesada (indica nombre, columnas clave y columnas incluidas) y volver a medir.
5. Redactar una conclusión de cinco a siete líneas comparando antes y después, y explicando si cambió el algoritmo de unión.

Salida esperada. Capturas del plan y de la pestaña Mensajes antes y después, una lista con lecturas lógicas por tabla, CPU y tiempo transcurrido, el algoritmo de unión de cada par de tablas y el operador más costoso. Tras crear el índice, las lecturas lógicas de la tabla de mayor volumen deberían bajar. Si la mejora es pequeña, justifícalo con el tamaño de las tablas.